Smart Housing Scout

Rent with evidence, not claims.

ASSIGNMENT 9: SOFTWARE PROJECT

Smart Housing Scout: a rental search that checks every claim

A web app for students who rent in Pittsburgh without seeing the home first. It ranks homes by real door-to-campus commute time, true monthly cost, and verified bathroom count, instead of what the listing says.

Name _____

Course _____

Date _____

Built with React, TypeScript, Vite, Tailwind CSS, Leaflet, Node and Express. Code written by Claude (Anthropic) from my prompt.

HOW THIS DOCUMENT IS MARKED

Grey boxes hold facts from the project: tables, numbers, and events that really happened. **Amber boxes marked YOUR TURN** are the parts the assignment requires in my own words, or as my own evidence. Screenshots were captured by Claude from the running app on Nov 9, 2026 and are labeled that way.

QUESTION 1

Functional decomposition (3 points)

The digital experience: an international student choosing an apartment in Pittsburgh, near Carnegie Mellon, before arriving in the city.

User objective (1 point)

FACTS TO WRITE FROM

- User: a student, often international, renting a home they cannot visit before signing.
- Goal: pick the home with the best real commute and real monthly cost.
- Why it matters: listings make claims ("5 minute walk to campus", "2 baths", "utilities included") that the student cannot check from abroad.

YOUR TURN. Write the objective in one or two sentences: the user, the goal, and why it matters.

Primary functions and system classification (2 points)

#	FUNCTION	DONE BY	FRONT OR BACK END	WHERE IT MEETS THE REAL WORLD
1	Choose the campus and the time class starts ("Arrive by")	User	Front end	The student's real class schedule
2	Search by neighborhood and set filters: budget, beds, baths, max commute, utilities	User	Front end	Keyboard, mouse, or touchscreen
3	Turn "8:45" into the next weekday at 8:45 Pittsburgh time	Program	Back end	The real calendar and the Eastern time zone
4	Get a transit route from each home to the campus building	Program	Back end (server)	Google Routes API, which uses real bus schedules. Sample model when no key is set
5	Split the route into walk, wait, ride, transfer, final walk, and early arrival; compute total minutes	Program	Back end (logic)	Real stop locations and departure times
6	Estimate true monthly cost: rent plus electricity, gas, water, internet, insurance	Program	Back end (logic, runs in browser)	Square footage, heating type, season

#	FUNCTION	DONE BY	FRONT OR BACK END	WHERE IT MEETS THE REAL WORLD
7	Compare listed baths with verified full and half baths; raise flags	Program	Back end (logic)	Tenant photos and floor plans
8	Show cards, map pins, Journey Bars, photos, tables, and plain-language findings	Program	Front end	The screen; OpenStreetMap tiles
9	Save homes and pick 2 to 4 to compare	User	Front end	Stored in the browser (localStorage)
10	Decide which home to rent and contact the landlord	User	Outside the software	A real lease and a real move

Note: the commute logic runs on a small Node server so the Google API key never reaches the browser. The cost, bathroom, and comparison rules run in the browser, but they are still back-end logic: they apply rules, while the page only displays results.

QUESTION 2A

Project description and architecture (6 points)

YOUR TURN. The assignment says this part may not be outsourced. Use the facts below and write each item in your own words. Keep the tables if you want; rewrite the sentences.

DEFINITION AND TARGET USER

Project: Smart Housing Scout, a rental search with a map and list side by side. Its difference is that every listing claim is checked and shown as evidence.

Target user: students and international students moving to Pittsburgh who have never visited. Default campus: Carnegie Mellon; Pitt and Duquesne also supported.

YOUR TURN. Definition and target user, in your words.

Requirements

#	REQUIREMENT	MET?
R1	Rank homes by door-to-campus commute time by default, with distance as secondary text	**Yes**
R2	Break each trip into walk, wait, ride, transfer, final walk, and early arrival, and draw it as a proportional Journey Bar	**Yes**
R3	Say in plain words when a closer home is slower ("Closer than Penn Ave, but 21 min slower")	**Yes**
R4	Show the true monthly cost with a toggle per utility and a season choice; label every estimate	**Yes**
R5	Flag a mismatch between listed and verified baths	**Yes**
R6	Show unstaged tenant photos, a before/after slider, and verified tenant reviews	**Yes**
R7	Compare 2 to 4 homes and highlight the best value in each row	**Yes**
R8	Keep the Google API key on the server; work fully offline with sample commutes when no key is set	**Yes, sample mode tested. Google mode not tested (no key)**

#	REQUIREMENT	MET?
R9	Loading, empty, and error states; keyboard focus; works on a phone	**Yes**

Inputs and outputs

INPUTS	OUTPUTS
Campus (CMU, Pitt, Duquesne) Arrive-by time (7:00 to 11:00 AM) Search text Filters: max budget, min beds, min verified full baths, max commute, utilities included Sort order Per-home: season, "included in rent" toggles	Commute: total minutes, each segment, bus lines, transfers, leave and arrive times True monthly cost and each utility estimate ($) Bath check: listed vs verified, mismatch flag Lease flexibility score (0 to 4) Tenant rating (1 to 5) Flags and plain-language findings

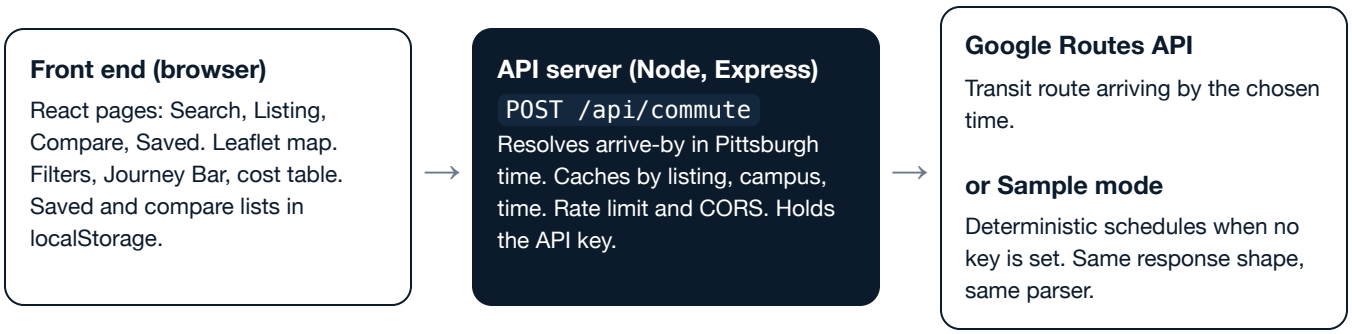
Fixed values

ITEM	VALUE
Campus buildings (where trips end)	CMU Gates Hillman Center; Pitt Cathedral of Learning; Duquesne Old Main
Time zone	America/New_York, computed on the server
Electricity ($ per sq ft per month)	Winter 0.085, summer 0.12, other months 0.07; electric heat adds 0.20 in winter
Gas ($ per sq ft per month)	Winter 0.17, summer 0.025, other months 0.06
Water and sewer; internet; insurance	$22 + $16 per bedroom; $65; $15
Building efficiency multiplier	Efficient 0.85, average 1.0, drafty 1.4 (on heating and cooling)
"Meaningful" commute difference	3 minutes or more

Logic

- **Departure:** the Routes API gives no trip start time, so departure = final arrival − route duration.
- **Wait** = bus departure − (departure + walk to stop). Transfer waits are added to the same bucket.
- **Early arrival** = arrive-by time − actual arrival at the building.
- **Total commute** = arrive-by time − departure. Early arrival counts, because the student spends it commuting.
- **True cost** = rent + every utility not included in rent. Annual average = mean of 4 winter, 4 summer, and 4 other months.
- **Bath check:** verified = full + 0.5 × half. Mismatch if it differs from the listed count.
- **Lease flexibility:** +1 lease under 12 months, +1 early termination allowed, +1 sublease (+0.5 with approval), +1 notice of 30 days or less.

Architecture



Shared code (`shared/`) is used by both sides: types, sample listings, the route parser, the cost model, and the comparison rules.

What success looks like

- The five built-in traps appear correctly (table below), on screen and in automated tests.
- Segment minutes always add up to the total commute.
- The cost table matches a hand calculation ($311 extra for a 1,000 sq ft, 2-bedroom, gas-heated home).
- When the server is off, the page says what happened and how to fix it, instead of breaking.

Plan for testing: the five traps

TRAP	HOME	EXPECTED RESULT (CMU, ARRIVE 8:45 AM)	RESULT
Closest home, slow commute	Semple St, 0.8 mi	39 min: 75 bus then 61C, arrives 14 min early. Not the fastest	**Pass**
Farther home, fast direct bus	Penn Ave, 2.3 mi	18 min on the 71D, no transfer. Fastest of 12	**Pass**
Listed 2 baths, really 1.5	Ellsworth Ave	Red mismatch flag: 1 full + 1 half	**Pass**
Cheap rent, high utilities	Greenfield Ave	Rent $950, true cost $1,405 (average), $1,560 in winter	**Pass**
Great reviews, long commute	Beechwood Blvd	Rating 4.8 of 5, commute 43 min (slowest)	**Pass**

Interface sketch

YOUR TURN. Draw the layout by hand and paste a photo here: top bar (logo, search, arrive-by), filter pills, list of photo cards on the left, map with price pins on the right.

QUESTION 2B

Working demonstration code (6 points)

1. Initial prompt

Submitted to Claude Code in one message. The full text is below, unchanged.

```
You are a senior product designer and full-stack engineer. Build "Smart Housing Scout", a rental search web app for students who rent sight-unseen. It should have the structure of Zillow (map and list side by side, filters, saved homes) and the craft of Airbnb (clean, calm, image-led, every detail finished). Its difference: every claim in a listing is checked and shown as evidence.

PRODUCT
- Name: Smart Housing Scout. Tagline: "Rent with evidence, not claims."
- Users: students and international students relocating to a city they have never visited. Default city: Pittsburgh. Default campus: Carnegie Mellon University. The campus selector includes CMU, University of Pittsburgh, and Duquesne University.
- Core idea: listings say "5 minute walk to campus", "2 bath", "utilities included", and show staged photos. Scout replaces each claim with something verified.

THREE SIGNATURE FEATURES
1. Commute Truth-Teller (the MVP; build it first and best)
 - Compares homes by true door-to-campus commute time, not distance.
 - For each listing, get a transit route from the listing's coordinates to the selected campus building, arriving by a chosen time (default 8:45 AM on the next weekday).
 - Break the trip into four parts: walk to stop, wait, ride, walk to building. Wait = transit departure time minus (trip start + walk-to-stop time). Show line names, number of transfers, and total minutes.
 - Signature visual: a proportional "Journey Bar", four colored segments scaled to minutes (walk teal, wait amber, ride blue, final walk light teal), with minutes labeled.
 - Default sort is "Fastest commute". Show distance as secondary text. Where the closest home is not the fastest, say so in plain words, for example "Closer than Maple St, but 11 min slower".
2. Lease Time Machine (shows what a listing hides), on the detail page as three tabs:
 - Real photos: unstaged, multi-angle room photos uploaded by past tenants, each with room label, angle, and date. Include a before/after slider that compares the listing photo with the unstaged photo of the same room. State clearly that photos are unstaged and not AI-edited.
 - True monthly cost: rent plus estimated electricity, gas, water and sewer, internet, and renters insurance, as a stacked bar and a table. Headline: "Listed $1,450. True monthly cost $1,687." A toggle for each utility lets the user mark it "included in rent". Estimates come from sample factors per square foot and season, labeled "estimate".
 - Past tenants: overall rating plus category ratings (landlord responsiveness, maintenance, noise, winter heating), and two or three short reviews with tenancy dates and a "Verified tenant" badge.
3. Bathroom Detective
 - Compare the listed bath count with the verified breakdown (full and half baths). If they differ, show a red mismatch flag with a one line explanation.
Also include a Lease terms panel (length options, deposit, early termination, sublease allowed, notice period) and a short accordion of GENERAL tenant-rights information, labeled "general information, not legal advice". Do not invent specific laws or statute numbers.

SCREENS
1. Search (home): sticky top bar with logo, one rounded search field (neighborhood, address, or campus), an "Arrive by" time picker, and filter pills (budget, beds, baths, max commute minutes, utilities). Below it, a split view. Left: scrollable result cards. Right: a map with price pins. Hovering a card highlights its pin, and clicking a pin scrolls to its card. On mobile: a map and list toggle with a bottom sheet.
2. Listing card: image carousel with dots and a heart (save), then price, beds, baths, square feet, address, a commute chip ("31 min door to campus") with a mini Journey Bar, and small badges ("Baths verified", "Unstaged photos", "True cost +$237").
3. Listing detail: Airbnb-style gallery (one large image and four small, with "Show all photos"), a
```

```
sticky right panel with the true monthly cost and Save, Compare, and Contact buttons, and sections for
Commute Truth-Teller, Lease Time Machine, Bathroom Detective, Lease terms, and neighborhood.
4. Compare (2 to 4 homes): a pivot table with homes as columns and rows for true commute (Journey Bars),
true monthly cost, baths (listed vs verified), lease flexibility, tenant rating, and flag count.
Highlight the best value in each row.
5. Saved homes.

DESIGN SYSTEM: Zillow's structure, Airbnb's finish
- Light, airy listing surfaces: white (#FFFFFF) cards over mist (#F6F8FA). Deep midnight navy (#0B1B2B)
for the header, nav, and the evidence panels. Evidence values (minutes, dollars, counts) use tabular
numerals or a monospace face, so verified data looks different from listing claims.
- Colors: primary action Scout green #0E9F6E, warning and hidden cost amber #F5A524, mismatch red
#E5484D, transit blue #2F6FED, hairline borders #E3E8EE. Do not copy Zillow's blue or Airbnb's red.
- Type: Inter for UI, tabular numerals for numbers, one display weight for headings. Scale 12 / 14 / 16
/ 20 / 28 / 40, line height 1.4 to 1.5, line length under 70 characters.
- Layout: 8 pixel grid. 16 to 20 pixel radius on images and cards, 10 to 12 on inputs, fully rounded
pills and buttons. Soft layered shadows only on hover and on floating elements. Generous whitespace.
Photos are the hero of every card.
- Motion: small and purposeful only. Card lift on hover, heart pop on save, carousel slide, Journey Bar
segments growing in once, skeleton loaders. Respect reduced-motion settings.
- States: loading skeletons, empty states with a clear next step, error states that say what happened
and how to fix it. Visible keyboard focus. WCAG AA contrast. Fully responsive.
- Copy: plain and specific. No marketing filler. No lorem ipsum.

DATA
- Use clearly labeled sample data: 12 fictional listings across Shadyside, Squirrel Hill, Oakland,
Bloomfield, Friendship, Point Breeze, and Greenfield, with realistic coordinates, rents ($900 to
$2,400), unit photos, utilities, reviews, and lease terms. Use royalty-free placeholder images with
stable IDs.
- Build in these test traps: one home that is closest but has a slow commute, one farther home with a
fast direct bus, one listing that says 2 baths but is verified 1 full and 1 half, one cheap rent with
high utilities, and one with excellent tenant reviews but a long commute.
- Define TypeScript types for Listing, CommuteResult (walkToStop, wait, ride, walkToBuilding, transfers,
lines, totalMinutes), CostBreakdown, TenantReview, and BathroomCheck.

TECH
- React with TypeScript, Vite, Tailwind CSS, and React Router. Leaflet with OpenStreetMap tiles for the
map, so no map key is needed.
- A small Node and Express backend that proxies Google's Routes API for transit: POST
https://routes.googleapis.com/directions/v2:computeRoutes with travelMode TRANSIT and arrivalTime, using
the headers X-Goog-Api-Key and X-Goog-FieldMask. Request only the fields needed
(routes.legs.steps.transitDetails, step durations, start and end times, total duration). Keep the API
key on the server in an environment variable, never in the front end. Provide a .env.example.
- Endpoint: POST /api/commute with origin, destination, and arriveBy. Cache results in memory by
listing, campus, and time. Add a simple rate limit and CORS.
- Mock mode: if no API key is set, return deterministic sample commutes and show a small "Sample commute
data" label, so the app works fully offline.
- Write unit tests for the function that turns a Routes API response into the four journey parts, using
a fixture JSON.

OUT OF SCOPE
User accounts, payments, real listing scraping, and the Zillow API.

HOW TO WORK
Build in this order: (1) design tokens and layout shell, (2) sample data and types, (3) search page with
map and cards, (4) Commute Truth-Teller with backend and mock mode, (5) listing detail with the Lease
Time Machine and Bathroom Detective, (6) compare page and saved homes, (7) polish and accessibility. Do
not ask me questions. Make reasonable assumptions and list them at the end. Do not claim anything works
until you have run it, and tell me exactly how to run it and where to add the Google API key.
```

REVIEW NOTES SENT WITH THE PROMPT (SUMMARY)

The same message included a review of the prompt. Claude applied each point:

- **Early arrival:** with "arrive by 8:45", the real loss is landing at 8:22. Added as a fifth segment; total = arrive-by − departure.
- **Transfers:** a trip with a transfer has two rides and two waits. The Journey Bar shows the trip as an ordered sequence.
- **Routes API field names:** check the docs. Claude confirmed that steps use `staticDuration` and that a route has no departure time.
- **Campus building** and **time zone** (America/New_York on the server) defined.
- Utility factors written out, photo pairs labeled illustrative, localStorage for saved homes, "best" defined per Compare row, tests beyond the parser, commit after each stage.

2. The finished interface

All screenshots: captured by Claude with headless Chrome from the running app, Nov 9, 2026, sample commute data.

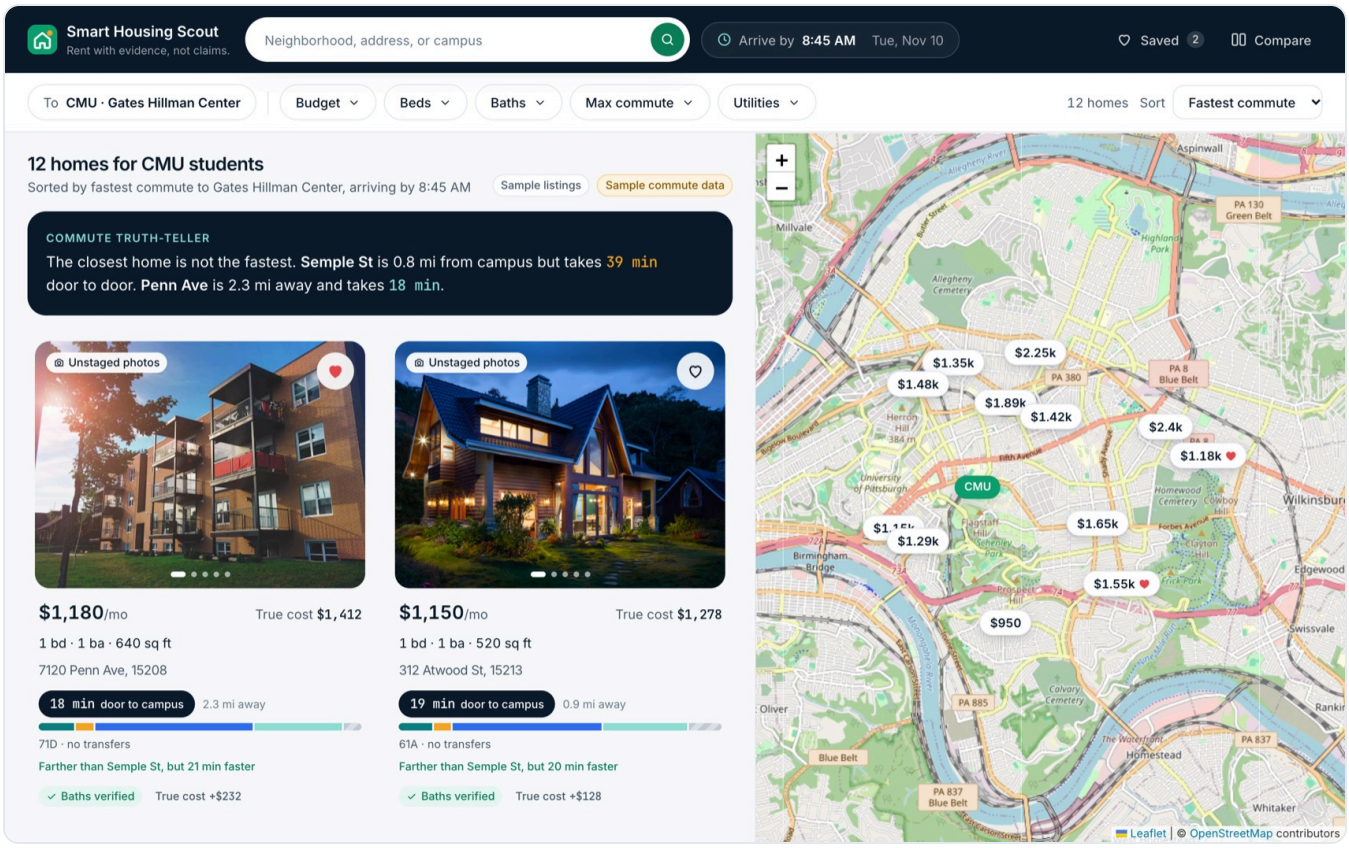


Figure 1. Search page. Cards sorted by fastest commute, map with price pins, and the finding "The closest home is not the fastest".

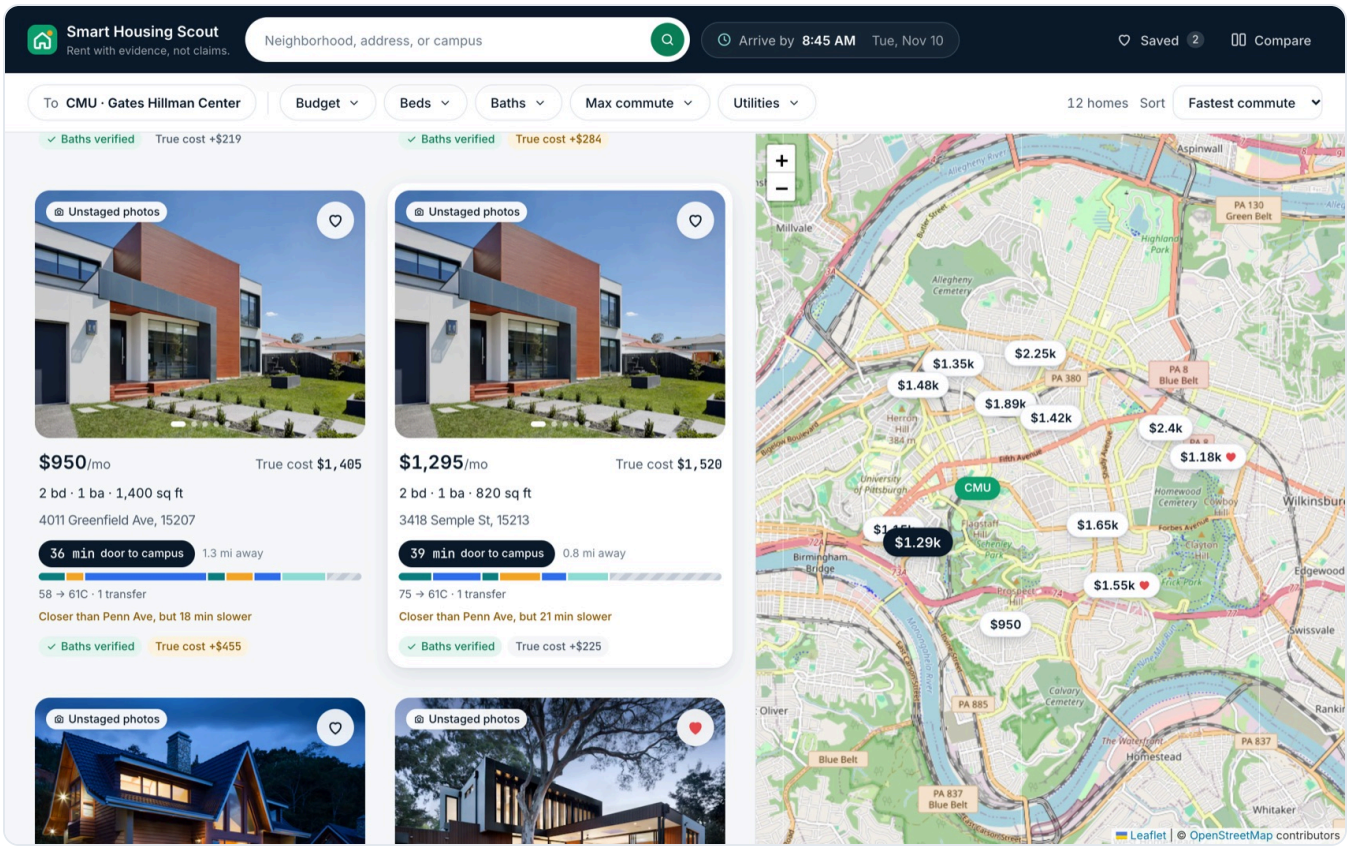


Figure 2. Hovering the Semple St card highlights its pin on the map ($1.29k, dark). Each card shows the mini Journey Bar and a finding such as "Closer than Penn Ave, but 21 min slower".

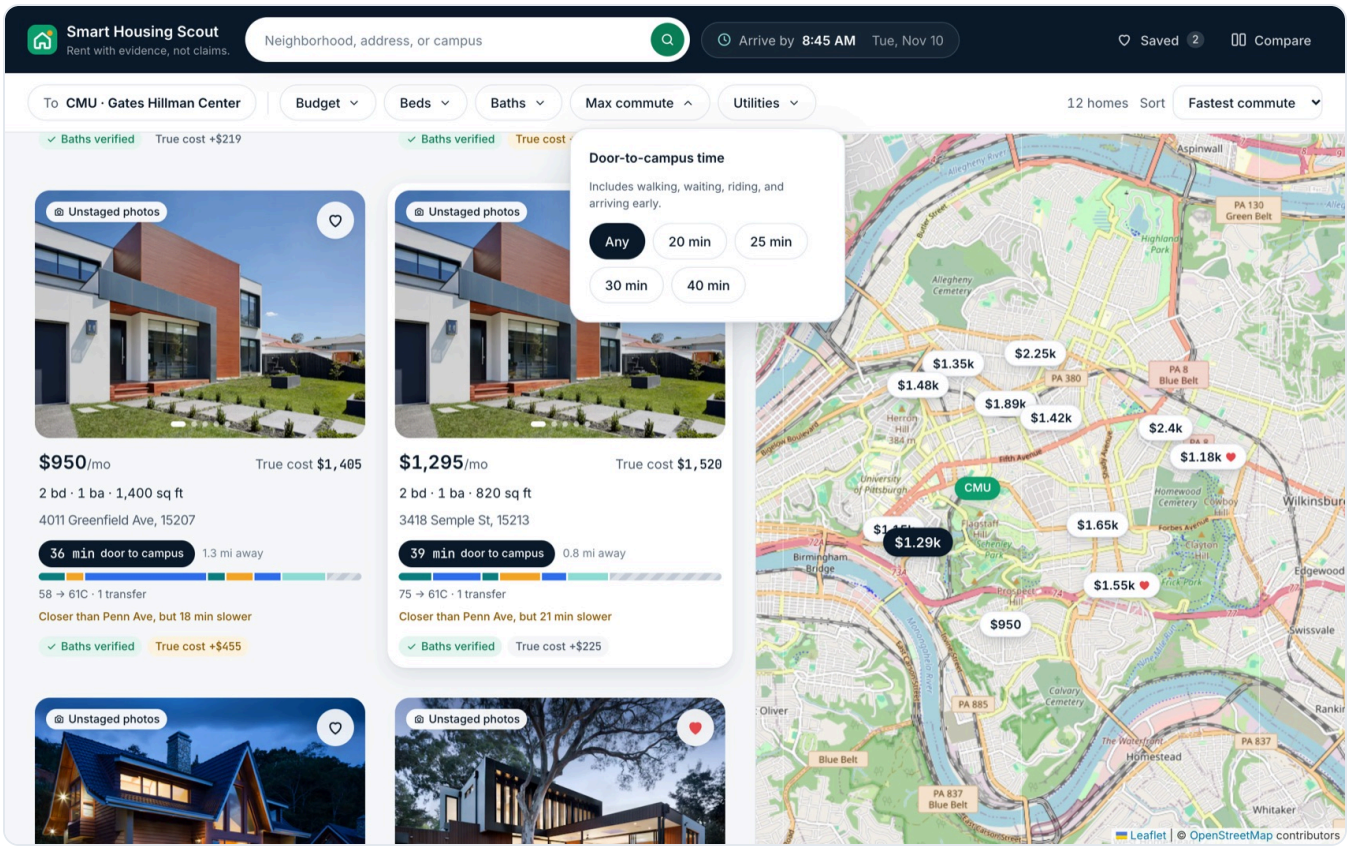


Figure 3. Filter pills. The Max commute filter counts walking, waiting, riding, and arriving early.

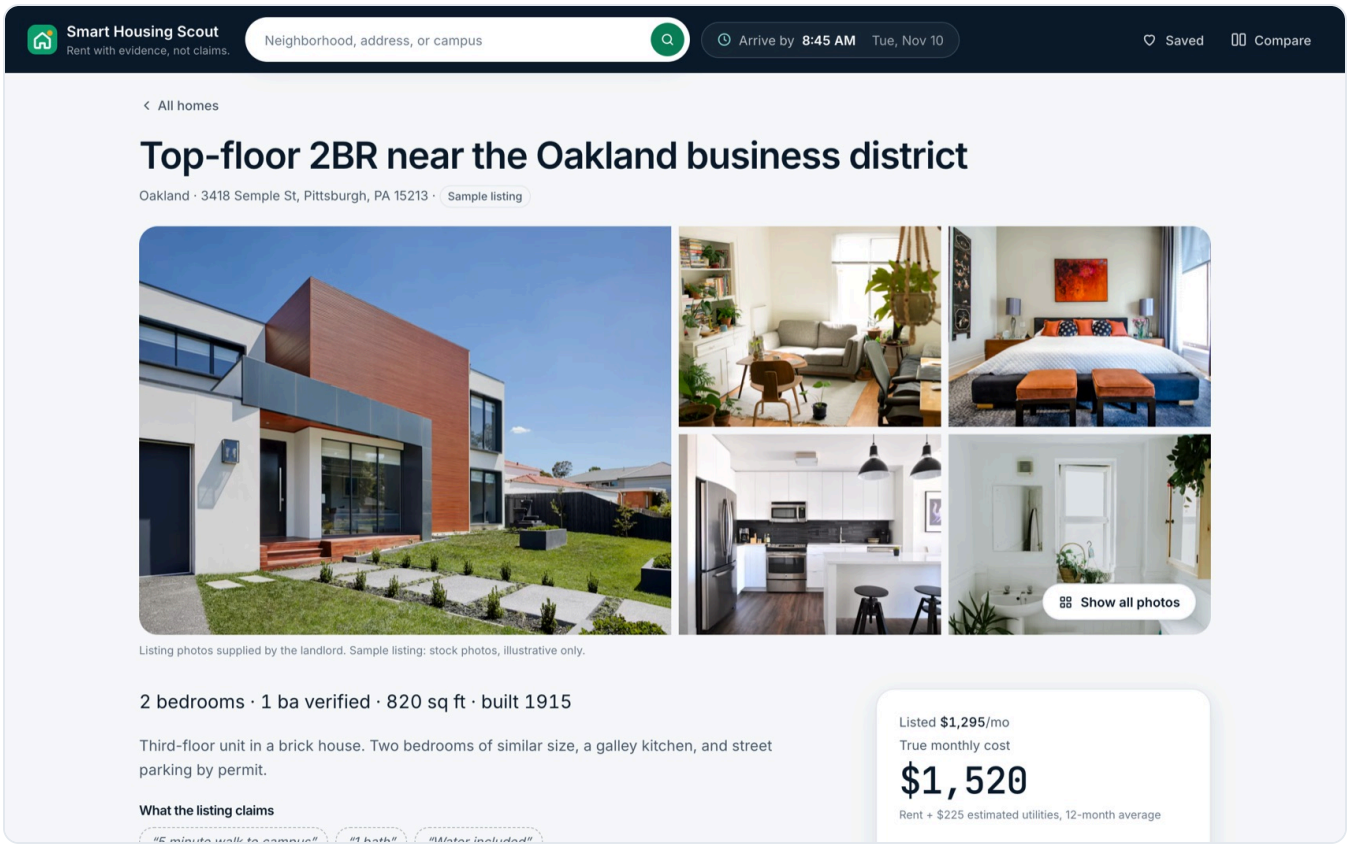


Figure 4. Listing detail: gallery (one large, four small), the listing's own claims, Scout's flag, and the sticky cost panel.

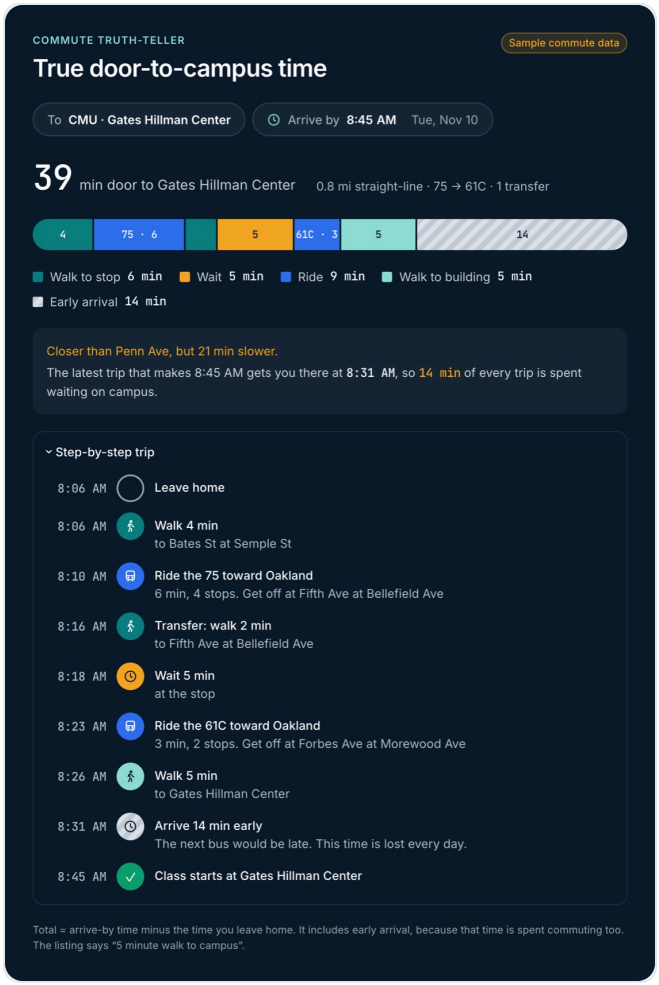


Figure 5. Commute Truth-Teller for the closest home. The listing says "5 minute walk to campus"; the real trip is 39 min with a transfer and 14 min of early arrival.

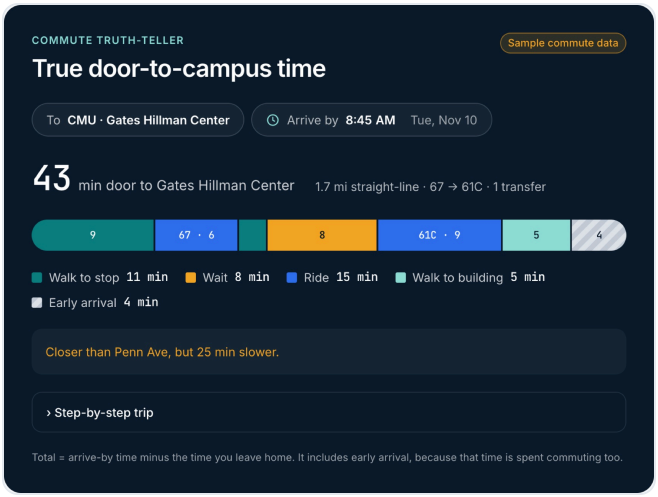


Figure 6. The best-reviewed home (Beechwood Blvd) has the longest commute: 43 min, infrequent 67 bus plus a transfer.

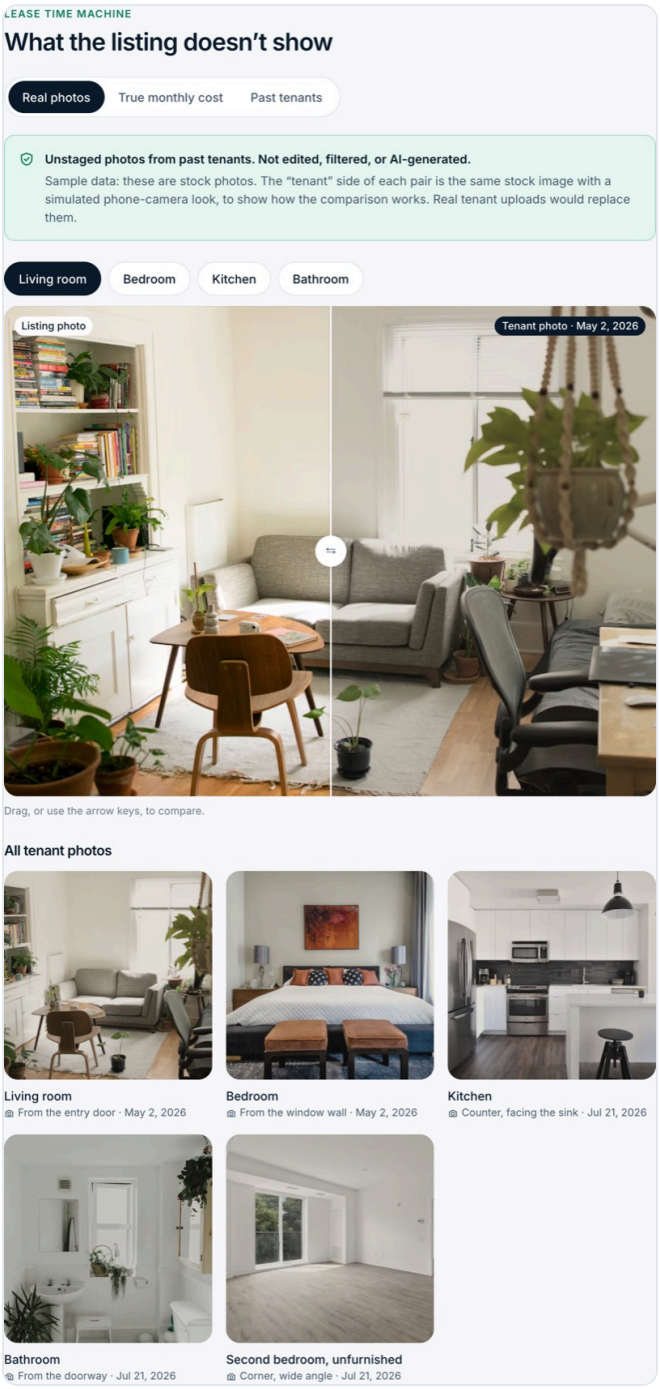


Figure 7. Lease Time Machine, Real photos tab: before/after slider and dated tenant photos, with an honest note that these are illustrative stock pairs.

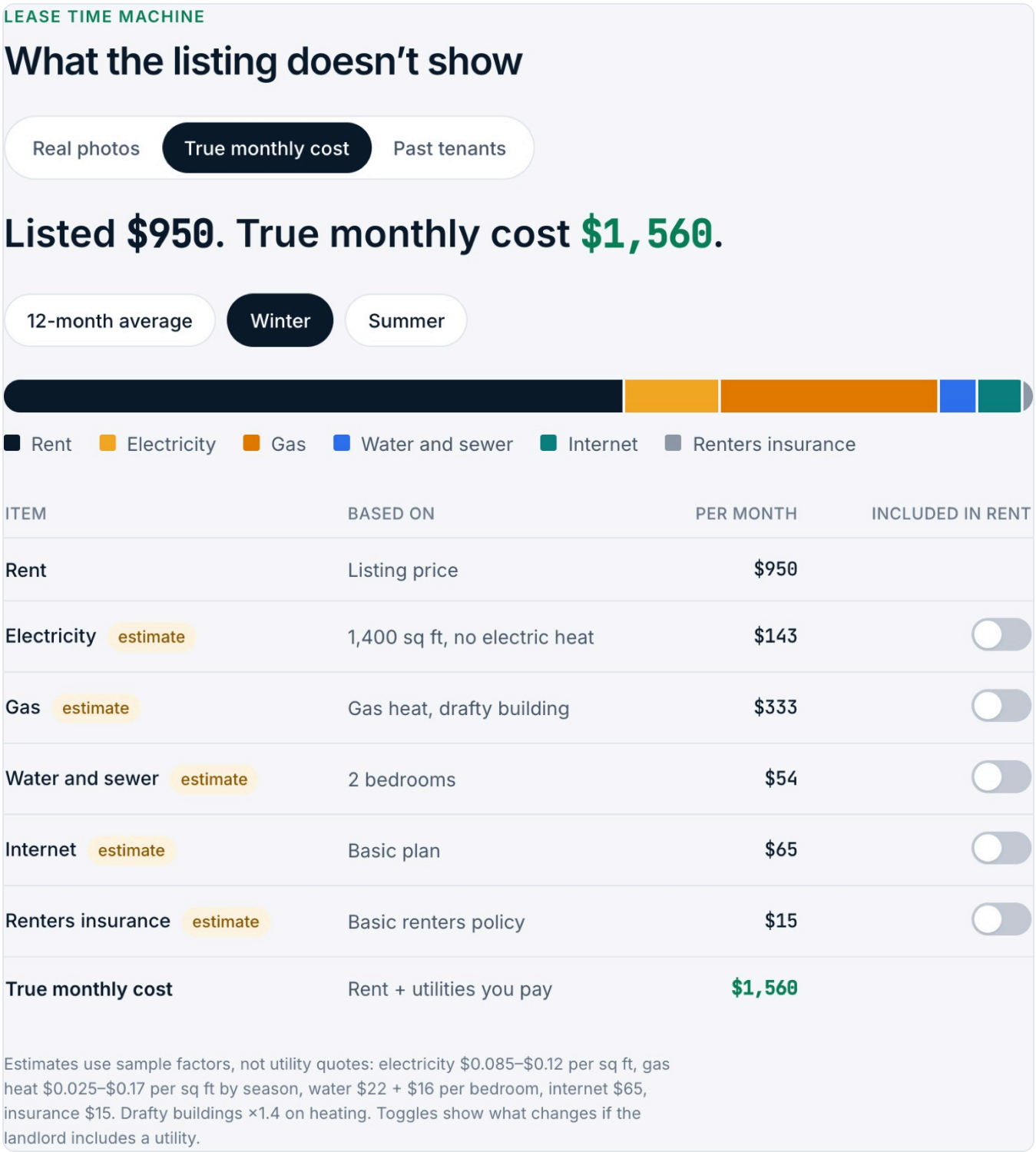


Figure 8. True monthly cost for the cheapest listing, winter: "Listed $950. True monthly cost $1,560." Gas alone is $333 in a drafty house.

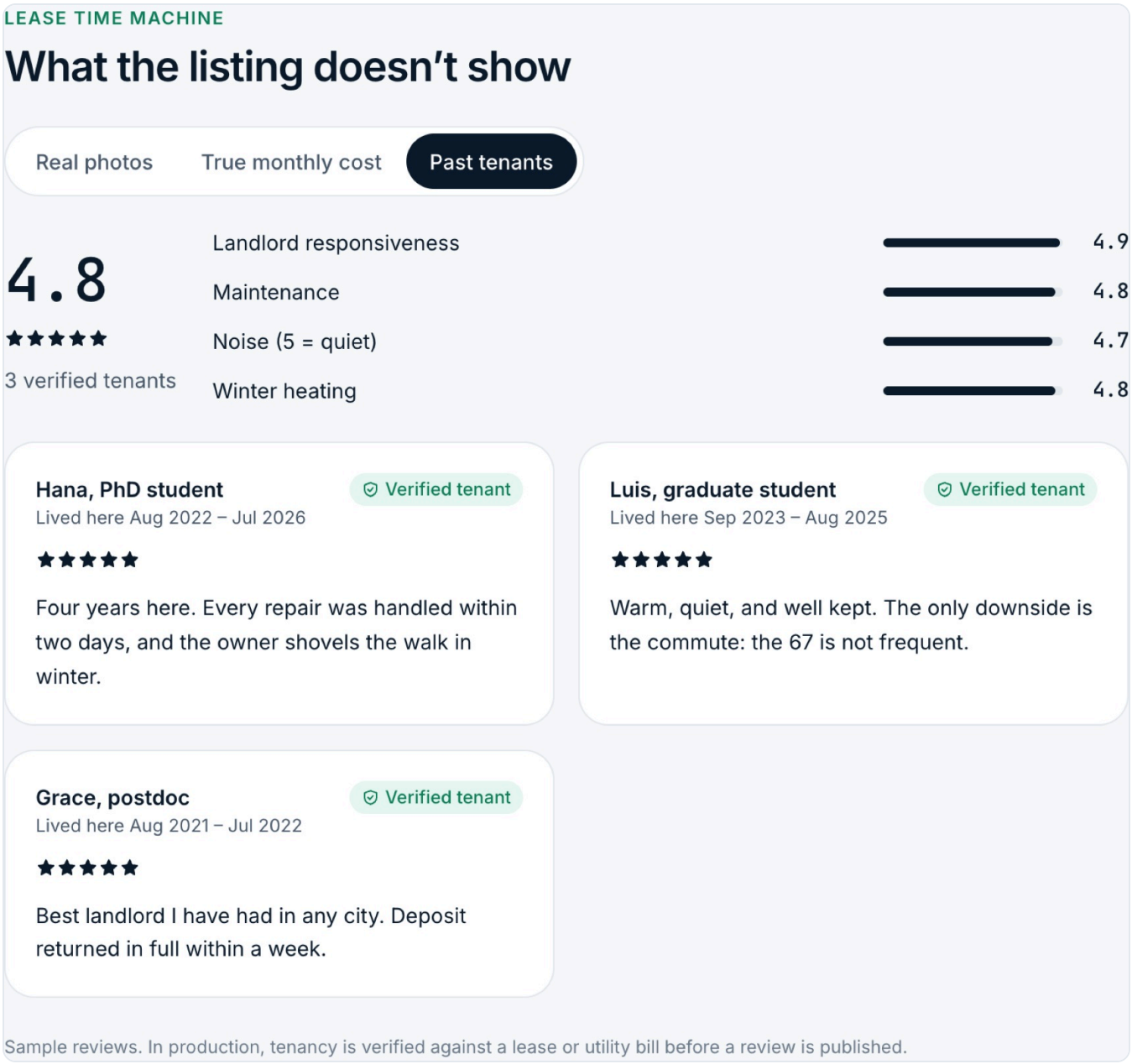


Figure 9. Past tenants tab: category ratings and reviews with a Verified tenant badge and tenancy dates.

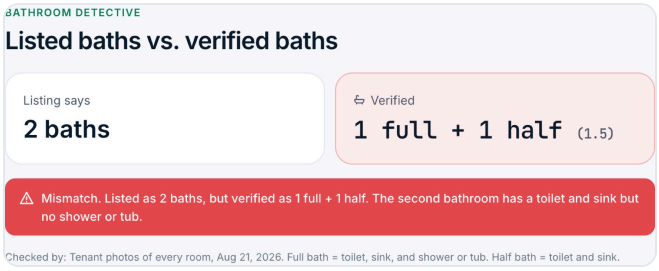


Figure 10. Bathroom Detective: listed 2 baths, verified 1 full + 1 half, red mismatch flag.

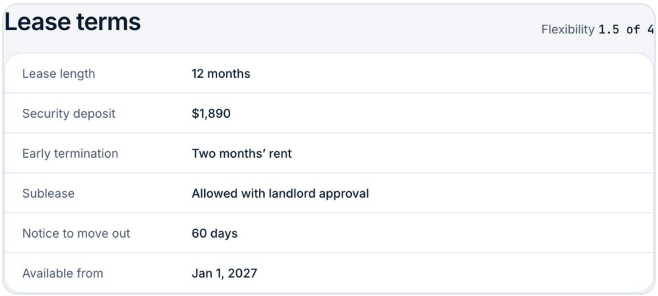


Figure 11. Lease terms with a flexibility score.

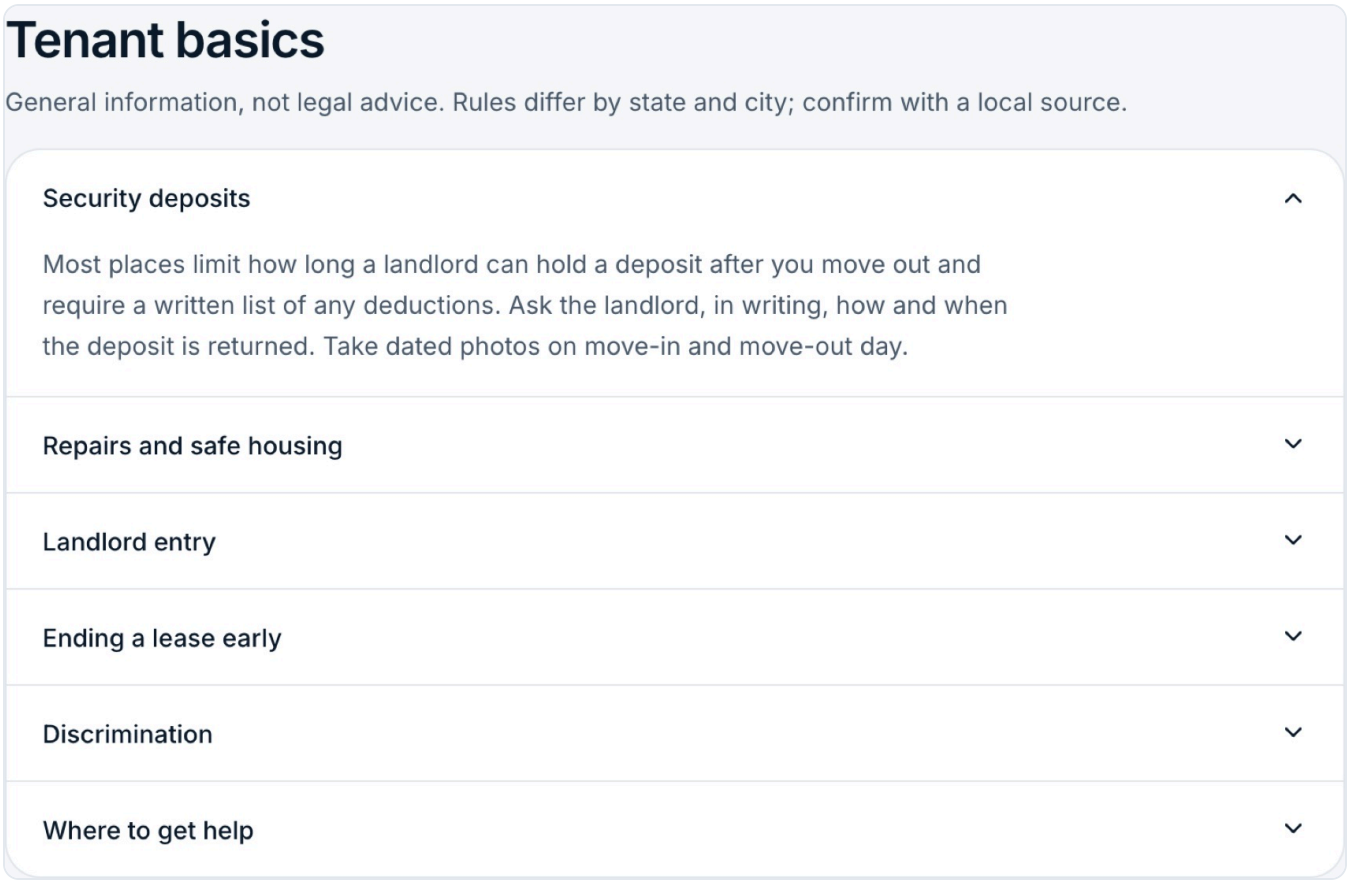


Figure 12. Tenant basics, labeled "general information, not legal advice". No statute numbers.

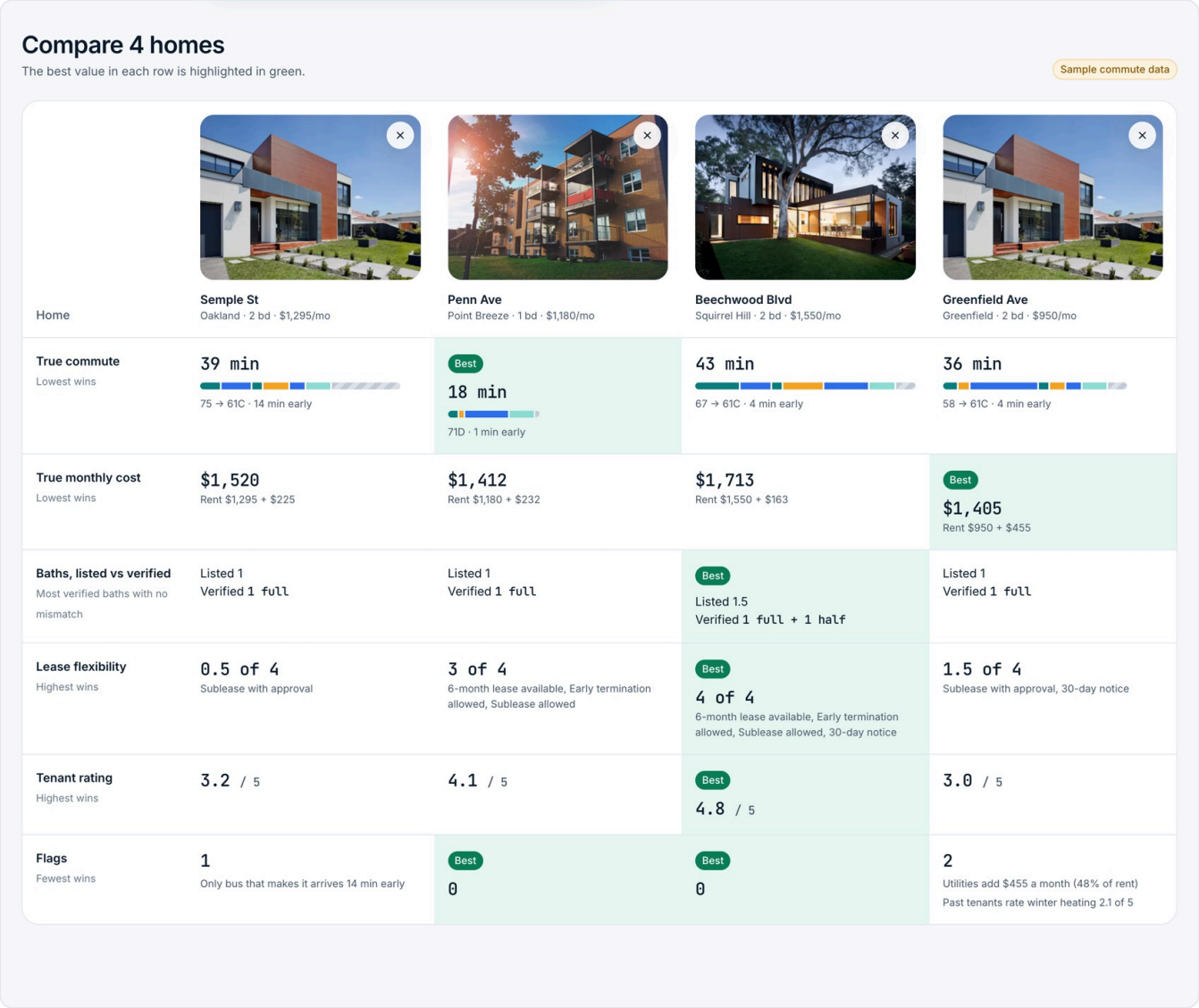


Figure 13. Compare 4 homes. The best value in each row is green: lowest commute and cost, most verified baths with no mismatch, highest flexibility and rating, fewest flags.

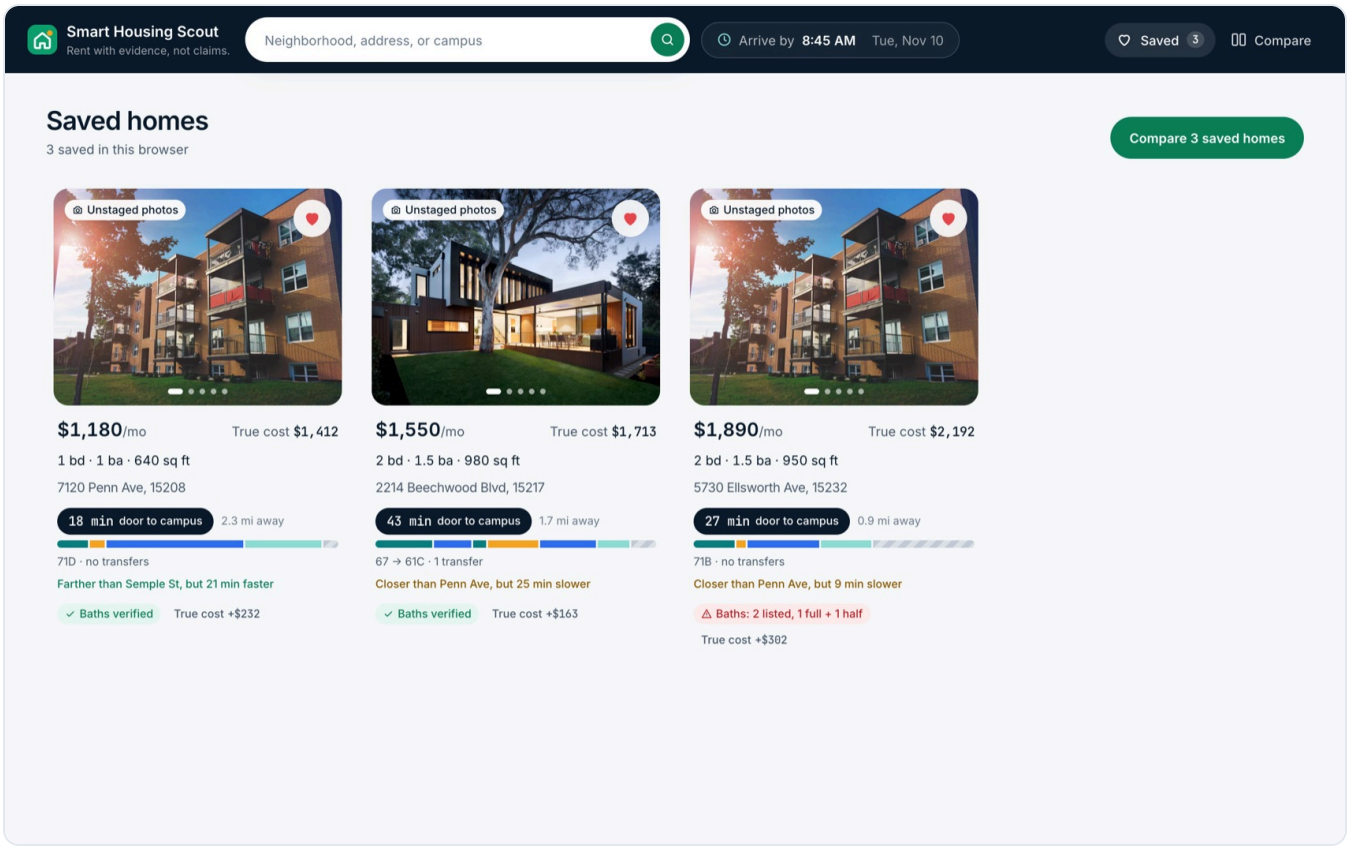


Figure 14. Saved homes, stored in the browser.

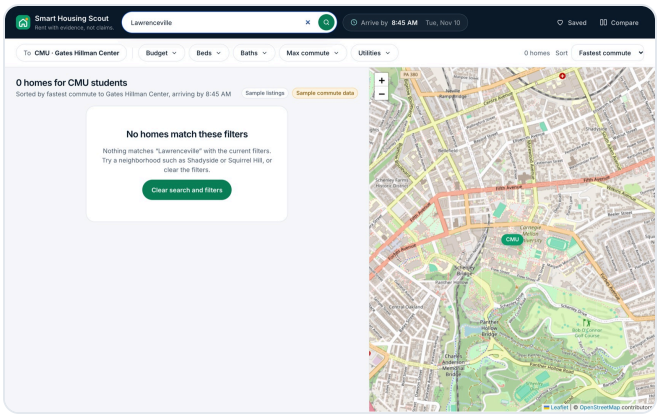


Figure 15. Empty state with a clear next step.

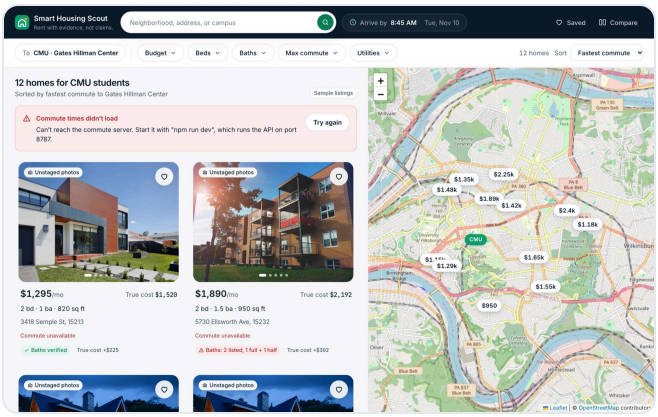


Figure 16. Error state when the commute server is off: what happened, how to fix it, and a Try again button.

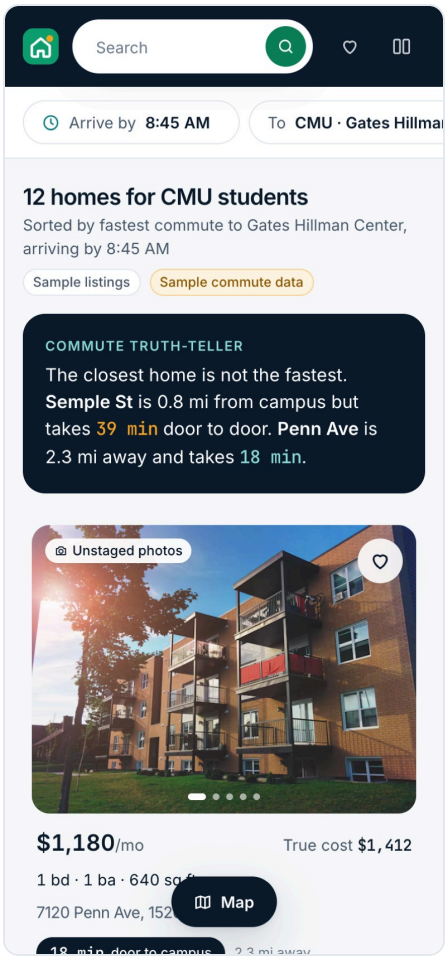


Figure 17. Phone: list view.

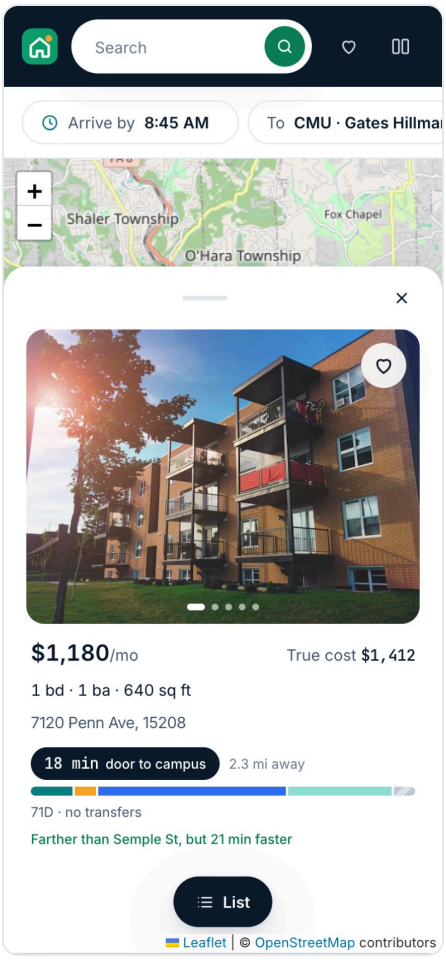


Figure 18. Phone: map view; tapping a pin opens a bottom sheet.

3. Testing verification

Automated tests, run with `npm test` on Nov 9, 2026. All 35 pass. The fixture is a Google Routes API response with one transfer.

```
✓ time: Monday afternoon -> Tuesday 8:45 EST (13:45 UTC)
✓ time: Monday early morning -> same day
✓ time: Friday evening -> Monday
✓ time: late Thursday night in Pittsburgh is still Thursday, even though it is Friday in UTC
✓ time: uses EDT in summer (12:45 UTC)
✓ time: rejects bad input
✓ time: formats clock times in Pittsburgh time
✓ parser: reads Google duration strings
✓ parser: derives departure from final arrival minus route duration
✓ parser: splits the trip into journey parts       (walk 5, wait 5, ride 13, transfer walk 2, final walk
4, early 10)
✓ parser: counts lines and transfers               (61B, 71A; 1 transfer)
✓ parser: defines total as arrive-by minus departure, including early arrival   (39 min)
✓ parser: keeps segments in trip order
✓ parser: assumes no initial wait when route duration is missing
✓ parser: never produces a negative initial wait if duration is too short
✓ parser: handles a walk-only route
```

```
✓ parser: rounds boundaries so odd seconds still add up
✓ parser: returns null when there is no route
✓ cost: matches a hand-calculated 12-month average  ($92 + $85 + $54 + $65 + $15 = $311)
✓ cost: excludes utilities the landlord includes
✓ cost: lets the user override what is included
✓ cost: costs more in winter for gas heat, and more for drafty buildings
✓ cost: total always equals rent plus non-included items
✓ cost: sample trap: cheapest rent is not the cheapest home
✓ insights: says when a closer home is slower than a farther one
✓ insights: says when a farther home is faster
✓ insights: ignores differences under 3 minutes
✓ insights: returns nothing when distance and time agree
✓ insights: names the closest and the fastest home
✓ insights: flags 2 listed vs 1 full + 1 half
✓ insights: accepts 1.5 listed as 1 full + 1 half
✓ insights: scores each flexibility feature
✓ traps: closest home is not the fastest; a farther direct bus wins
✓ traps: best-reviewed home has the longest commute
✓ traps: is deterministic

Test Files  4 passed (4)
     Tests  35 passed (35)
```

Manual checks in a browser: toggling "Internet included" lowered the Ellsworth Ave total from $2,192 to $2,127 (–$65); hovering a card highlighted its pin; the API rejected `"8.45"` as an arrive-by time with a clear message; a request from another website got no CORS permission.

YOUR TURN. If your instructor wants your own evidence, run `npm run dev`, repeat the five trap checks, and replace or add your own screenshots here.

QUESTION 2C

Code modification (2 points)

Changes made after the first working version, found while testing in a real browser. Pick one to describe.

CHANGE	WHY	HOW
A. Photo labels fixed	The first screenshot showed a house with palm trees as a Pittsburgh listing, and some "bedroom" photos were kitchens. Code tests could not catch this.	Every stock photo was viewed on a contact sheet and re-sorted into living room, kitchen, bedroom, bath, and exterior lists in `shared/listings.ts`. Exteriors were spread so neighboring cards differ.
B. "Farther but faster" compares with the closest home	The Penn Ave card said "Farther than Westminster Pl, but 25 min faster", which compared it with a random slow home. The useful comparison is with the home a renter would pick by distance.	In `commuteInsight()`, slower homes are now sorted by distance, not by time. The card now reads "Farther than Semple St, but 21 min faster".
C. Clear message when the server is off	When the API restarted, the dev proxy answered with an empty 500 error, so the page would have said "Commute server returned 500", which does not tell the user what to do.	In `fetchCommute()`, any 500-level response without a JSON body now shows: "Can't reach the commute server. Start it with npm run dev" (Figure 16).
D. Phone layout	On a 375 px phone the "Arrive by" pill wrapped onto two lines and the search placeholder was cut to "Neighbc". The compare tray covered the map's bottom sheet.	Pills got `whitespace-nowrap`; the placeholder becomes "Search" on narrow screens; the tray hides in map mode.

WHO DID WHAT

Claude found these problems while testing in the browser, wrote the code changes, and re-ran the tests.

YOUR TURN. Choose one change. Describe what changed, why, and how, in your own words. Say truthfully who typed the code. If you make your own change, describe that one instead.

QUESTION 2D

Summary reflection (3 points)

TRUE EVENTS TO DRAW FROM

Systems approach

- The prompt defined the users, the five test traps, the data types, the build order, and an offline mode before any code was written. The traps worked as a test plan: each one has an expected result that can be checked.
- A review of the prompt before building found the biggest gap: "arrive by 8:45" hides time lost to arriving early. That became the fifth segment and changed how total commute is defined.

Plan compared with the finished project

- Node.js was not installed on the laptop, so Claude downloaded the official version for the session only.
- The traps did not appear on their own. Claude had to adjust the sample bus schedules so the closest home really was slow and the far home really was fastest, and added tests so they stay that way.
- The Google Routes API has no trip start time, which the prompt assumed. The parser works it out from the route duration instead.

Working with the AI

- Most helpful: building a full app with a server, tests, and offline mode from one detailed prompt, and checking the API documentation before writing code.
- Where it needed checking: screenshots caught problems that 35 passing tests missed: wrong photos, a confusing comparison, and phone layout bugs.
- The rate limit worked: the screenshot script itself was blocked after 300 requests a minute, which proved the limit is real.
- Not yet verified: the real Google transit path, because no API key was available.

Next time

- Write expected numbers for each trap in the prompt, not only descriptions.
- Sketch the phone layout too, not only desktop.
- Get an API key early so real data can be tested.

YOUR TURN. Write several paragraphs in your own words: your systems approach, plan vs. result, how you worked with the AI, and what you would do differently.

QUESTION 3

Group review (5 points)

FROM YOUR NOTES

Gautam's project: a poker game with several game modes, such as a biggest-hand game. Clear interface with strong colors and contrast, very interactive, with a list of past battles. It felt like a full website, not just an assignment.

Your examples about the AI: fast code and tests from a defined prompt; screenshots caught problems the tests missed (photos, comparison wording, phone layout).

YOUR TURN. Use only what happened in your group. Cover: the feedback the group gave on Smart Housing Scout; challenges everyone shared; what you learned from how others built and prompted (for example Gautam); where the AI helped, where it was hard, and what you would do differently.

Before you submit

- □ Fill in every YOUR TURN box in your own words.
- □ Add the photo of your hand-drawn sketch (Question 2A).
- □ Fill in name, course, and date on the cover.
- □ Check that the statement of who did what is accurate.
- □ Upload to Canvas. Optional: attach the project folder or a link to the code.